

郑 先生

AI 协同全栈开发者 | Next.js / tRPC / TypeScript

37 岁 | 电话: __18664314673__ |

GitHub: <https://github.com/zhengzhao1176>

在线 Demo: <https://ent-ops.pages.dev> (演示账号 admin / Aa123456)

个人简介

37 岁, 工厂普工转行的全栈开发者。2023 年起通过 ChatGPT 自学编程, 目前以 Claude Code 为主力工具, 专注 AI 协同的 Web 系统交付。我擅长的不是手写代码量, 而是把 AI subagent 当成团队来调度——由我负责拆分业务需求、设计技术规约 (spec)、定义 agent 读写边界与质量门禁, 由 18~26 个 subagent 并行实现各模块, 最终通过完整测试矩阵验收。已用这种方法独立交付多个企业级 Web 系统, 单个项目从 BRD 到上线约 2 天。

核心优势

AI 工程化交付能力: 能把 BRD → 技术 spec → 多 agent 并行实现 → 测试全绿 → 部署上线, 跑成可复现的流水线, 而不只是 prompt 拼代码。

测试驱动质量意识: 项目均带完整 4 层测试金字塔 (unit / property / component / E2E), 最高覆盖到 Stryker 变异测试, 把『agent 不审 src, 机器全绿即视为正确』作为信任契约。

全栈独立闭环: 从 Prisma schema、tRPC 契约、状态机、审计落地, 到前端表单、Antd 组件、Playwright E2E、Cloudflare 部署, 全部可独立完成。

极强的自学与执行力: 从工厂普工岗位起步, 2 年时间走到能独立交付带边缘部署 + 完整测试矩阵的企业级系统。

对自身定位清晰: 清楚自己手写底层算法不是强项; 强项是用 AI 把想法快速落地为带测试、可上线、可演进的产品。

技术栈

前端框架: Next.js 14 / 15 (App Router)、React 18、Vue 3、TypeScript (strict)

UI 与样式: Antd 5、Tailwind CSS 3、@tanstack/react-query 5

后端与契约: tRPC v11、Zod、Node.js、JWT、bcryptjs

数据层: Prisma 5、SQLite、Cloudflare D1、@prisma/adapters-d1

测试体系: Vitest 2、fast-check (property)、Testing Library 16、Playwright 1.46、Stryker 8 (变异测试)

部署与基建: Cloudflare Pages + D1 + Workers、Edge Runtime、Web Crypto API

AI 工作流: Claude Code、subagent 编排 (最多 26 并行)、文档驱动 TDD、读写边界 hooks

工程基础: Git / GitHub、pnpm、CI 流水线、命令矩阵 / 质量门禁

项目经历

GitHub: <https://github.com/zhengzhao1176> (下列项目均使用 Claude Code + subagent 协同独立完成)

项目一 ent-ops · 企业基础运营管理系统

时间 / 周期: 2026.05 | 从 BRD 到上线约 2 天

技术栈: Next.js 14 + tRPC 11 + Prisma 5.22 + Cloudflare D1 + Tailwind CSS

在线 Demo: <https://ent-ops.pages.dev> (admin / Aa123456)

项目角色: 独立开发者 (人写 BRD / spec / task 规约, subagent 实现)

覆盖用户管理、角色权限、部门、商品、入库、出库、调拨、盘点、审计 9 大模块的企业管理系统。

设计并落地 18 份 subagent 任务规约 (domain-modeler → contract-writer → repo-impl → service-impl → router-impl → ui-impl → e2e → bug-fixer → flake-detective), 跑出可复现的 TDD pipeline。

测试矩阵: 52 单元 + 158 集成 + 21+ Playwright E2E, 全部绿色提交。

针对 Cloudflare D1 边缘运行时做了多项工程化处理: Lazy Prisma Proxy 避免 Workers 初始化报错; runInTransaction helper 在本地走 \$transaction、在 D1 上退化为顺序执行; 用 Web Crypto API 替代 node:crypto 跨 Node 18+ / Workers / Edge 运行时。

Inbound / Outbound / Transfer / Stocktake 四套业务状态机集中声明式维护, 每个写操作 service 末尾自动落审计日志 (before / after JSON)。

亮点: 仓库不仅交付了运行的系统, 还沉淀了《通用工作流-CRUD 模式驱动框架》方法论文档与全套 task 规约——把『AI 协同开发流程』做成了可复现的工程实证。

项目二 库存管理系统 · 文档驱动 × 26 subagent 并行交付

时间 / 周期: 2026.05 | 约 2 天交付

技术栈: Next.js 15 + tRPC v11 + Prisma 5 + SQLite + Antd 5 + React 18

项目角色: 独立开发者 (人审 BRD / spec / tests, agent 实现 src)

覆盖认证、分类、供应商、商品、采购单、销售单、库存统计 7 大模块的全栈库存管理系统。

设计 26 个 subagent 协作矩阵 (7 模块 × 3 角色 + 5 全局 agent), 仅通过文件系统协作, 由 .claude/settings.json hooks 强制执行读写边界 (test-agent 严禁读 src/, impl-agent 严禁写 tests/)。

建立 4 层测试金字塔 + 双门禁: 75 unit + 17 property (fast-check 30 random runs/property) + 44 component (Testing Library + userEvent) + 34 真 Chromium E2E (playwright-cli 风格 persistent context), 均在 CI 全绿。

落地 Stryker 8 变异测试做最终验收 (mutation kill ≥ 95%), 把『测试看起来全绿』和『测试真的能抓到 bug』分开度量。

解决了一系列 Next 15 + Antd 5 真实坑点: Antd Modal.confirm 静态 API 不渲染 (改用 App.useApp().modal)、Antd 自动 CJK 间隔破坏 testid、Select aria-label 重复挂载需自定义 helper、Next 15 Link 不再允许 <a> 子元素等, 全部沉淀进 issues 文档。

亮点: 把『人写 spec、agent 不审 src、机器全绿即视为正确』做成显式的信任契约, 是对 AI 协同开发流程的方法论尝试。

AI 协同开发方法论

我做这件事的方式跟『用 AI 写代码』不太一样，更接近『用 AI 跑工程』：

需求层：人写 BRD（业务唯一来源）+ 技术 spec（test-agent 的唯一输入），保证业务意图不丢失。

调度层：把模块拆成 domain / contract / repo / service / router / ui / e2e 等角色，每个角色一份 subagent 任务规约。

边界层：用 Claude Code 的 hooks 强制读写边界（实现 agent 不能改测试，测试 agent 不能读实现），杜绝『agent 改测试让自己绿』。

验收层：4 层测试金字塔 + 变异测试 + E2E，机器全绿即视为 src 正确——人只审小而慢变化的 spec / tests。

复现层：所有 task 规约、命令矩阵、CI 模板都进仓库，整套流程可被任何拉到这个 repo 的人完整复现。

自学与转型经历

2023 年：开始通过 ChatGPT 自学编程，从 HTML / JavaScript / Node.js 基础起步。

2023 – 2024 年：系统学习 Vue 3、Node.js、TypeScript，独立完成入门到进阶的练手项目。

2024 – 2025 年：转向 React / Next.js + tRPC + Prisma 全栈方向，开始用 Claude Code 作为主力开发工具。

2026 年至今：把工作方式从『AI 辅助单人编码』升级到『subagent 矩阵编排 + 文档驱动 TDD』，持续在公开仓库输出方法论与实证。

自我评价

我清楚自己的传统编码功底属于一般水平——徒手写算法、抠框架内核不是我的强项。我的强项是把 AI 当成团队来用：能把一个业务需求拆成 spec、拆成 agent、拆成测试，再让 AI 在严格的边界里把它跑成可上线、可演进的产品。从工厂普工到能独立交付带 4 层测试矩阵 + 边缘部署的企业级系统，是我用两年时间自学走出来的路。我希望加入一个真心愿意拥抱 AI 协同开发的团队，把这种交付速度用在真实的业务问题上。